PPH7095F - WMXJOL001

Jolene Wium

2026-04-02

Biostatistics III Assignment

Table of contents

Plagiarism Declaration iii

AI Tools Declaration iv

Marital Satisfaction Analysis 1

Part 1: Descriptive data analysis . . . 1

Question 1 . . . 1

Question 2 . . . 1

Question 3 . . . 1

Part 2: Generalized estimating equations (GEEs) . . . 1

Question 4 . . . 3

Question 5 . . . 3

Question 6 . . . 3

Part 3: Linear mixed effects (LME) model . . . 4

Question 7 . . . 4

Question 8 . . . 4

Question 9 . . . 5

Part 4: Model validation . . . 5

Question 10 . . . 5

Conclusion . . . 5

Appendix A: Python code for analysis 7

Plagiarism Declaration

I acknowledge that plagiarism is a serious form of academic misconduct and dishonesty.

I have read and understood the document, 'Avoiding Plagiarism: A guide for students' from the University of Cape Town. I acknowledge that I understand that the utilisation of open AI tools (such as ChatGPT) without proper acknowledgement and citation is also a form of academic misconduct and is not allowed.

I acknowledge and accept the institutional penalties that are applied if I should plagiarise in any submitted work. I have been warned that for coursework assessment: first-time and minor misconduct breaches will result in a mark deduction and warning; and any repeat or major misconduct breaches will result in a failing grade and possible escalation to the Faculty Misconduct Committee (at the course convenor's discretion).

AI Tools Declaration

I declare that in the preparation of this assignment, I used ChatGPT (OpenAI GPT-5) as a support tool. The tool was used for assistance with:

1. Clarifying statistical concepts (e.g., model interpretation, model fit metrics),
2. Refining Python code snippets.
3. Formatting mathematical expressions and tables in Quartro.

All statistical analyses, interpretation of results, and final critical reflections were conducted by me. The AI tool did not generate findings, nor did it substitute for my own analysis, understanding, or judgment. I have reviewed and edited the outputs to ensure accuracy, appropriateness, and alignment with the course requirements.

Marital Satisfaction Analysis

Part 1: Descriptive data analysis

Question 1

Table 1 presents the baseline characteristics of the individuals in the dataset, stratified by sex. Males are on average 2.2 years older than females when first getting amarried. Females in the study population earned on average more than males. The majority of both sexes cohobted before marriage (both > 85%). Females reported higher household workloads than males, with males taking on only 6.2% of the household workload on average, compared to 62.1% for females.

Table 1: *Baseline characteristics stratified by sex.*

	Female	Male
Age at marriage	27.31 (4.70)	29.59 (4.85)
Income	32502.84 (18539.35)	30976.49 (16881.02)
Cohabitation		
> 0	42 (14.7%)	25 (12.0%)
> 1	243 (85.3%)	184 (88.0%)
Housework allocation		
> 0	108 (37.9%)	196 (93.8%)
> 1	177 (62.1%)	13 (6.2%)

Note: Continuous variables are presented as *mean (SD)*; categorical variables as *n (%)*.

Question 2

Figure 1 depicts the mean longitudinal profile of satisfaction measurements by household workload. The plot shows that individuals with higher household workloads tend to have lower marital satisfaction over time, while those with lower household workloads tend to have higher marital satisfaction. This suggests a potential negative association between household workload and marital satisfaction, which may warrant further investigation.

Question 3

Figure 2 shows the marital satisfaction score over time for each individual over time. The left hand graph depics satisfaction of individuals that chose not to cohabitate with their partners before getting married. The right hand shows those that did co-habitate before getting married.

In general, both groups show a clear downward trajectory over time with significant indivudual variation. The overall patterns appear similar across cohabitation groups.

This section evaluated visual trends in marital satisfaction over time. The next section will evaluate general population level trends in the data using Generalized Estimating Equations (GEEs).

Part 2: Generalized estimating equations (GEEs)

This section used `statsmodels` version 0.14.6 package in a Python environment (version 3.13.1). Two GEE models were fitted using the `GEE` function. The AR(1) model was fitted with the `Autoregressive` covariance structure, while the unstructured model was fitted with `Unstructured` covariance structure. Robust (sandwich) standard errors were used in both models, as is the default for this package.

Figure 1: *Mean longitudinal profile of satisfaction by household workload.*

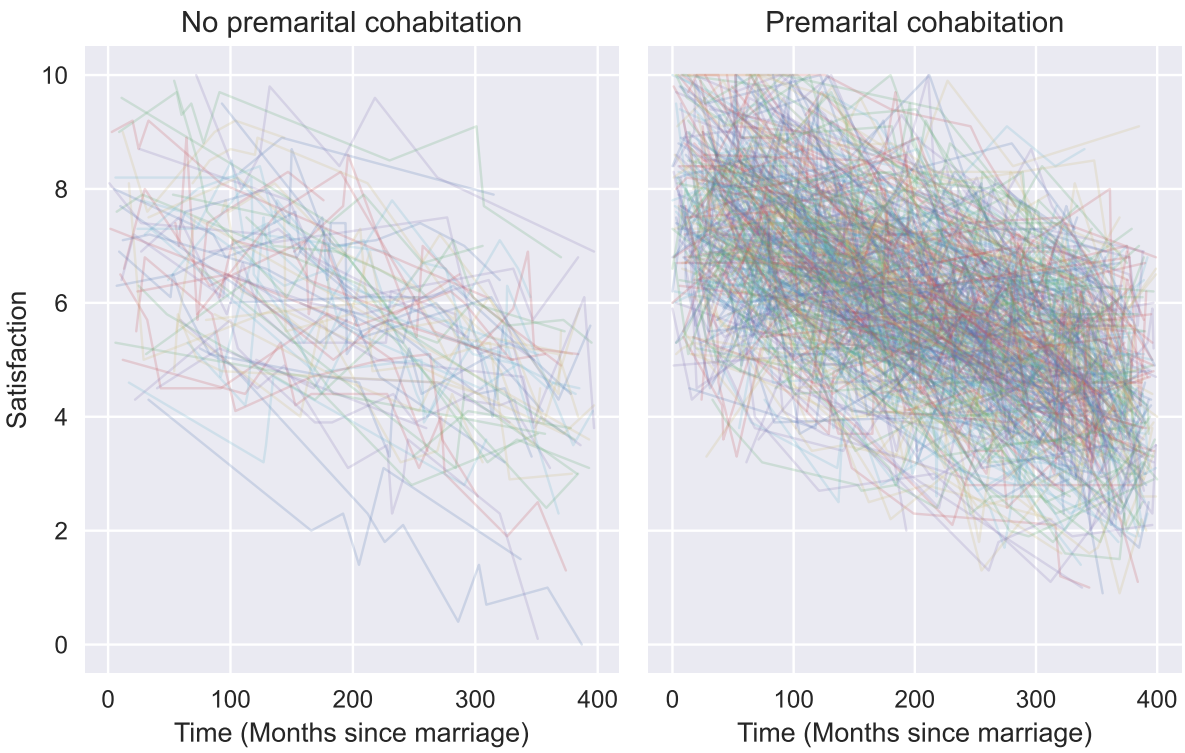

Figure 2: *Spaghetti Plot of Satisfaction by Premarital Cohabitation.*

In GEE 1 (AR1), the `time` argument was set to `obs_time`, whereas in GEE 2 (the model using the unstructured covariance structure) used a `visit` count variable to represent the observation order for each individual and `obs_time` in the model to still represent the time variable. This is a requirment of the package and can be explored in the source code of the function published on GitHub[1].

GEE 2 (Unstructured) was fitted excluding the last two observations of individual *2191*, the only individual with more than 15 observations. This was necessary to accomodate restrictions in the covariance structure. The model fitted well with 3 811 observations (as oppose 3 813).

Question 4

Table 2 presents the estimated associations between marital satisfaction, time and covariates from two GEE models with different working correlation structures. Overall, the model results are very similar. This suggests that the results are stable with some sensitivity to the correlation structure.

Table 2: *GEE model estimates with robust standard errors and 95% confidence intervals.*

	AR(1)		Unstructured	
	Estimate (SE)	95% CI	Estimate (SE)	95% CI
Intercept	7.543 (0.366)	6.825 to 8.261	7.599 (0.886)	5.863 to 9.335
Male (vs Female)	-0.403 (0.144)	-0.685 to -0.122	-0.748 (0.372)	-1.478 to -0.019
Time	-0.009 (0.000)	-0.009 to -0.008	-0.009 (0.000)	-0.009 to -0.008
Age at marriage	0.014 (0.011)	-0.008 to 0.037	0.022 (0.024)	-0.026 to 0.070
Premarital cohabitation	0.260 (0.163)	-0.060 to 0.580	-0.087 (0.388)	-0.847 to 0.673
Income	-0.000 (0.000)	-0.000 to 0.000	0.000 (0.000)	-0.000 to 0.000
Household workload	-0.470 (0.144)	-0.752 to -0.189	-0.471 (0.238)	-0.937 to -0.006

Question 5

The two models, resulting from different correlation structures, produce very similar coefficient estimates. This indicates that the results are not highly sensitive to the choice of correlation structure. Some variation is observed in the width of the confidence intervals, with the AR(1) model generally producing narrower intervals and smaller standard errors.

The unstructured model required restricting the data to fewer time points to accommodate one 'group' (individual) with more than 15 observations, whereas the AR(1) model was able to use all available observations.

The AR(1) model is preferred as it is simpler, allows for the full use of the data, and is consistent with the expectation that observations closer in time are more strongly correlated than those further apart.

Question 6

Time: In the AR(1) model, the estimated coefficient for time is -0.009 (95% CI: -0.009 to -0.008). On average, marital satisfaction decreases by 0.009 for every month that the respondents are married, adjusting for other variables. The confidence interval does not include zero, providing evidence that time is a significant predictor for marital satisfaction (or dissatisfaction).

Premarital cohabitation: In the AR(1) model, the estimated coefficient for premarital cohabitation is 0.260 (95% CI: -0.060 to 0.580). On average, individuals who cohabited before marriage have 0.26 marital satisfaction units higher compared to those who did not, while adjusting for other

[1] https://github.com/statsmodels/statsmodels/blob/main/statsmodels/genmod/cov_struct.py#L231

variables. However, the confidence interval includes zero, indicating statistical significant evidence of an association between premarital cohabitation and satisfaction.

Household workload: In the AR(1) model, the estimated coefficient for household workload is -0.470 (95% CI: -0.752 to -0.189). On average, individuals with higher household workloads have 0.47 marital satisfaction units lower compared to those with lower workloads, while adjusting for other variables. The confidence interval does not include zero, providing statistical significant evidence of a negative association between household workload and marital satisfaction.

This section used GEE models to evaluate population-level associations between marital satisfaction and predictors. The next section will use linear mixed effects models to evaluate both population-level and individual-level associations, accounting for individual variation in trajectories over time.

Part 3: Linear mixed effects (LME) model

This section used `statsmodels` version 0.14.6 package in a Python environment (version 3.13.1). The LME models were fitted using the `MixedLM` function, using maximum likelihood estimation.

Question 7

Table 3 provides a comparison of the fixed-effect coefficient estimates, 95% confidence intervals, and model comparison metrics for two LME models with different random-effects structures. LME 1 includes only random intercepts, while LME 2 includes both random intercepts and random slopes for time, allowing for individual variation in both baseline satisfaction and the rate of change in satisfaction over time. Model fit statistcs are provided at the bottom of the table for each model.

Overall, Log-likelihood, AIC and BIC indicate lower values for LME 2 than LME 1.

Table 3: *Comparing covariance matrixes for LME models.*

	LME 1: Random intercept		LME 2: Random intercept + slope	
	Estimate (SE)	95% CI	Estimate (SE)	95% CI
Intercept	7.567 (0.376)	6.831 to 8.303	7.568 (0.378)	6.828 to 8.309
Male (vs Female)	-0.431 (0.135)	-0.695 to -0.166	-0.435 (0.135)	-0.701 to -0.170
Time	-0.009 (0.000)	-0.009 to -0.008	-0.009 (0.000)	-0.009 to -0.008
Age at marriage	0.012 (0.011)	-0.010 to 0.034	0.012 (0.011)	-0.010 to 0.034
Premarital cohabitation	0.218 (0.157)	-0.090 to 0.526	0.215 (0.158)	-0.094 to 0.525
Income	-0.000 (0.000)	-0.000 to 0.000	-0.000 (0.000)	-0.000 to 0.000
Household workload	-0.496 (0.134)	-0.758 to -0.233	-0.494 (0.134)	-0.757 to -0.231

Note: Fixed effects are presented as estimate (SE) with 95% confidence intervals.
Model fit statistics:
> LME 1 AIC: 10654.57, BIC: 10710.79, Log-likelihood: -5318.28;
> LME 2 AIC: 10637.76, BIC: 10706.47, Log-likelihood: -5307.88.

Question 8

LME 2 is the preferred model. LME 2 is a nested version of LME 1, adding random slopes for time. Fixed effect estimates are similar in both models, indicating consistency across models. However, LME 2 has higher log-likelihood and lower AIC and BIC values than LME 1, indicating a better fit to the data. The random slope variance in LME 2 is non-zero, suggesting between-individual variation in time trends, however small.

This result is supported by the spagetti plot (Figure 2) which showed clear downward individual trajectories over time with variation between individuals.

Question 9

Time: In both GEE 1 (AR(1)) and LME 2 (random intercepts & random slopes) model, the estimated coefficient for time is -0.009 (95% CI: -0.009 to -0.008). On average, marital satisfaction decreases by 0.009 for every month that the respondent is married, adjusting for other variables. The confidence interval in both models do not include zero, providing evidence that time is a significant predictor for marital satisfaction (or dissatisfaction).

Premarital cohabitation: In the AR(1) model, the estimated coefficient for premarital cohabitation is 0.260 (95% CI: -0.060 to 0.580), whereas in the LME 2 model it is 0.215 (95% CI: -0.094 to 0.525). On average, individuals who cohabited before marriage have slightly higher marital satisfaction compared to those who did not. However, the confidence interval includes zero in both models, indicating no strong evidence of an association between premarital cohabitation and satisfaction.

Household workload: In the AR(1) model, the estimated coefficient for household workload is -0.470 (95% CI: -0.752 to -0.189), whereas in LME 2 it is -0.494 (95% CI: -0.757 to -0.231). On average, individuals with higher household workloads have lower marital satisfaction compared to those with lower workloads, adjusting for other variables. The confidence interval does not include zero, providing strong evidence of a negative association between household workload and marital satisfaction.

Part 4: Model validation

Question 10

Figure 3 illustrate the residuals plot for the final LME model (LME 2). The plot shows a random scatter of residuals around zero, approximately symmetrically distributed. This suggests that the assumption of homoscedasticity is satisfied. A small number of outliers and slight variation in spread are observed at higher fitted values, these are not substantial enough to suggest an incorrectly defined model.

Figure 4 shows the QQ plot of the random intercepts and random slopes for the final LME model (LME 2). The points approximately follow the reference line, suggesting that the random intercepts are approximately normally distributed. Some deviation from normality is observed at the tails.

Conclusion

This assignment evaluated the association between marital satisfaction and time, premarital cohabitation, and household workload using both GEE and LME models. The results suggest that marital satisfaction decreases over time, is negatively associated with household workload, and shows no strong evidence of an association with premarital cohabitation. The LME model with random slopes provided a better fit to the data than the model with only random intercepts, indicating individual variation in trajectories of marital satisfaction over time.

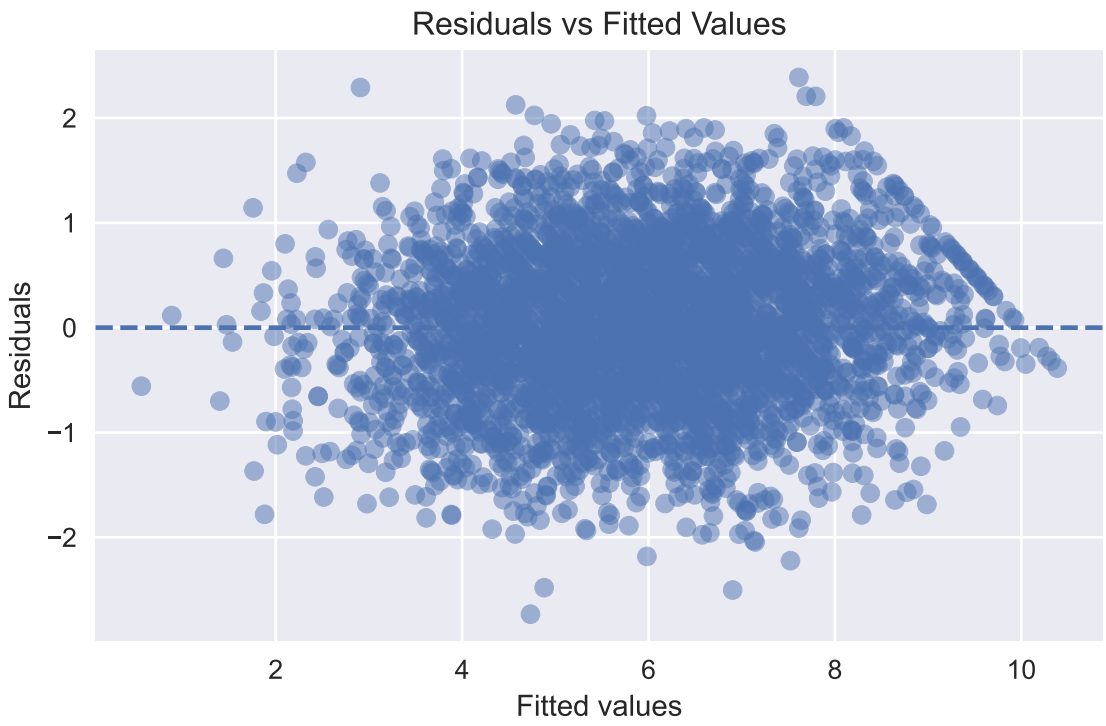

Figure 3: *Residuals Plot for Final LME Model - LME 2 Random Intercepts and Slopes.*

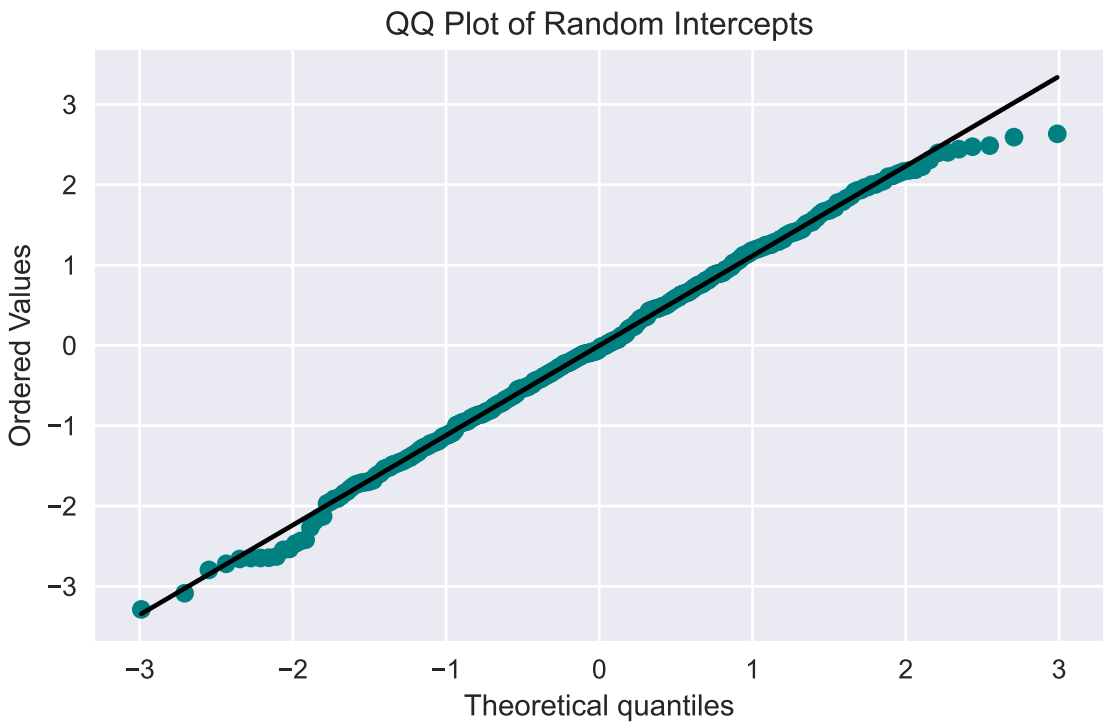

Figure 4: *QQ Plot of Residuals for Final LME Model - LME 2 Random Intercepts and Slopes.*

Appendix A: Python code for analysis

The full reproducible analysis pipeline is provided below.

```
import os

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import scipy.stats as stats
import statsmodels.api as sm
import statsmodels.formula.api as smf
from IPython.display import Markdown, display
from statsmodels.genmod.cov_struct import Autoregressive, Unstructured, Independence
from statsmodels.genmod.families import Gaussian

import config


DATA_PATH = config.DATA_PATH

def load_data(data_path=DATA_PATH):
    """Load dataset and sample 70% of individuals while preserving full profiles."""
    if not os.path.exists(data_path):
        raise FileNotFoundError(f"Data file not found at {data_path}")

    df = pd.read_csv(data_path)

    # Set seed
    np.random.seed(1987090901)

    # Sample 70% of unique IDs
    unique_ids = df["id"].unique()
    sample_size = int(0.7 * len(unique_ids))
    sampled_ids = np.random.choice(unique_ids, size=sample_size, replace=False)

    # Keep full longitudinal profiles
    df = df[df["id"].isin(sampled_ids)].copy()

    # Set obs_time as integer
    df["obs_time"] = df["obs_time"].astype(int)

    # Sort df (visit order within person) and create visit variable
    df = df.sort_values(["id", "obs_time"]).reset_index(drop=True)
    df["visit"] = df.groupby("id").cumcount()
```

```
    # Keep relevant columns
    columns = [
        "id",
        "obs_time",
        "satisfaction",
        "sex",
        "age_marriage",
        "cohab",
        "income",
        "hw_all",
        "visit",
    ]

    df = df[columns]

    return df


# ##############################################################################
# PART 1: Descriptive data analysis
# ##############################################################################
# -----------------------------
# Question 1: Baseline characteristics
# -----------------------------
def summarize_cont(df, var, by="sex", digits=2):
    """Summarize continuous variables as mean (SD)."""
    out = df.groupby(by)[var].agg(["mean", "std"])
    return out.apply(
        lambda row: f"{row['mean']:.{digits}f} ({row['std']:.{digits}f})",
        axis=1,
    )


def summarize_cat(df, var, by="sex", digits=1):
    """Summarize categorical variables as n (%)."""
    counts = pd.crosstab(df[var], df[by], dropna=False)
    perc = (
        pd.crosstab(df[var], df[by], normalize="columns", dropna=False) * 100
    )
    return counts.astype(int).astype(str) + " (" + perc.round(digits).astype(str) + "%)"


def add_categorical_block(table_rows, df, var, block_name, by="sex"):
    """Append categorical variable block to Table 1."""
    cat_summary = summarize_cat(df, var, by=by)
```

```
    # Header row
    header_row = {"Variable": block_name}
    for col in cat_summary.columns:
        header_row[str(col)] = ""
    table_rows.append(header_row)

    # Category rows
    for level in cat_summary.index:
        row = {"Variable": f" > {level}"}
        for col in cat_summary.columns:
            row[str(col)] = cat_summary.loc[level, col]
        table_rows.append(row)


def table1_descriptive(df):
    """Create Table 1: baseline characteristics stratified by sex."""

    # Baseline dataset (first observation per individual)
    df_baseline = (
        df.sort_values(["id", "obs_time"])
        .groupby("id", as_index=False)
        .first()
    )

    # Ensure consistent column order for sex
    sex_levels = sorted(df_baseline["sex"].dropna().unique())

    table1_rows = []

    # Continuous variables
    continuous_vars = [
        ("Age at marriage", "age_marriage"),
        ("Income", "income"),
    ]

    for label, var in continuous_vars:
        summary = summarize_cont(df_baseline, var)
        row = {"Variable": f"{label} "}
        for sex in sex_levels:
            row[str(sex)] = summary.get(sex, "")
        table1_rows.append(row)

    # Categorical variables
    add_categorical_block(table1_rows, df_baseline, "cohab", "Cohabitation")
    add_categorical_block(
        table1_rows, df_baseline, "hw_all", "Housework allocation"
```

```
    )

    return pd.DataFrame(table1_rows)


# ----------------------------
# Question 2: Mean longitudinal profile of satisfaction
#             measurements by household  workload
# ----------------------------
def plot_mean_profile(df, n_bins=20):
    plt.close("all")

    plot_df = df[["obs_time", "hw_all", "satisfaction"]].dropna().copy()
    plot_df["time_bin"] = pd.cut(plot_df["obs_time"], bins=n_bins)

    summary = (
        plot_df.groupby(["time_bin", "hw_all"], observed=False)
        .agg(
            mean_satisfaction=("satisfaction", "mean"),
            sd_satisfaction=("satisfaction", "std"),
            n=("satisfaction", "size"),
        )
        .reset_index()
    )

    summary["time_mid"] = summary["time_bin"].apply(lambda x: x.mid)
    summary["se"] = summary["sd_satisfaction"] / np.sqrt(summary["n"])
    summary["lower"] = summary["mean_satisfaction"] - 1.96 * summary["se"]
    summary["upper"] = summary["mean_satisfaction"] + 1.96 * summary["se"]

    plt.style.use("seaborn-v0_8")
    fig, ax = plt.subplots(figsize=(7, 4.5))

    for hw in sorted(summary["hw_all"].dropna().unique()):
        subset = summary[summary["hw_all"] == hw].sort_values("time_mid")
        ax.plot(
            subset["time_mid"],
            subset["mean_satisfaction"],
            linewidth=2,
            label=f"{hw}",
        )
        ax.fill_between(
            subset["time_mid"],
            subset["lower"],
            subset["upper"],
            alpha=0.2,
```

```
        )

    ax.set_xlabel("Time (Months since marriage)")
    ax.set_ylabel("Mean satisfaction")
    ax.set_title("Mean longitudinal profile of satisfaction by household workload")
    ax.legend(title="Household workload")
    ax.grid(True, alpha=0.3)

    fig.tight_layout()


# ----------------------------
# Question 3: Spaghetti plot of satisfaction
#             measurements by prematital cohabitation
# ----------------------------
def plot_spaghetti(df):

    plt.style.use("seaborn-v0_8")
    ids = df["id"].unique()
    plot_df = df[df["id"].isin(ids)].copy()

    fig, axes = plt.subplots(1, 2, figsize=(7, 4.5), sharey=True)

    for i, cohab_value in enumerate(sorted(plot_df["cohab"].dropna().unique())):
        subset = plot_df[plot_df["cohab"] == cohab_value]

        for pid in subset["id"].unique():
            person = subset[subset["id"] == pid].sort_values("obs_time")

            axes[i].plot(
                person["obs_time"],
                person["satisfaction"],
                alpha=0.2,
                linewidth=1
            )

        if cohab_value == 0:
            cohab_label = "No premarital cohabitation"
        else:
            cohab_label = "Premarital cohabitation"

        # axes[i].set_title(f"Cohabitate: {cohab_value}")
        axes[i].set_title(f"{cohab_label}")
        axes[i].set_xlabel("Time (Months since marriage)")

    axes[0].set_ylabel("Satisfaction")
```

```
    plt.tight_layout()


# ##########################################################################
# PART 2: Generalized estimating equations (GEEs)
# ##########################################################################
def fit_gee_models(df):
    dat = df.dropna().copy()

    formula = (
        "satisfaction ~ obs_time + sex + age_marriage + "
        "cohab + income + hw_all"
    )

    # AR(1): all visits
    gee_ar1 = smf.gee(
        formula=formula,
        groups="id",
        time="obs_time",
        data=dat,
        family=Gaussian(),
        cov_struct=Autoregressive(grid=True),
    ).fit()

    # Unstructured: restricted visits
    dat_un = dat[dat["visit"] <= 14].copy()
    gee_un = smf.gee(
        formula=formula,
        groups="id",
        time="visit",  # visit order, but obs_time is used in the formula
        data=dat_un,
        family=Gaussian(),
        cov_struct=Unstructured(),
    ).fit()

    return gee_ar1, gee_un


def format_model(result):
    df_out = pd.DataFrame({
        "Variable": result.params.index,
        "Estimate (SE)": [
            f"{est:.3f} ({se:.3f})"
            for est, se in zip(result.params, result.bse)
        ],
        "95% CI": [
```

```
            f"{l:.3f} to {u:.3f}"
            for l, u in result.conf_int().values
        ],
    })
    return df_out.set_index("Variable")


def question2_pretty(df):
    gee_ar1, gee_un = fit_gee_models(df)

    ar1 = format_model(gee_ar1)
    un = format_model(gee_un)

    # Rename inner columns
    ar1.columns = ["Estimate (SE)", "95% CI"]
    un.columns = ["Estimate (SE)", "95% CI"]

    # Create MultiIndex columns
    ar1.columns = pd.MultiIndex.from_product(
        [["AR(1)"], ar1.columns]
    )
    un.columns = pd.MultiIndex.from_product(
        [["Unstructured"], un.columns]
    )

    table = pd.concat([ar1, un], axis=1)

    # Reset index to bring Variable back
    table = table.reset_index()

    # Rename variables
    rename_map = {
        "Intercept": "Intercept",
        "sex[T.Male]": "Male (vs Female)",
        "obs_time": "Time",
        "age_marriage": "Age at marriage",
        "cohab": "Premarital cohabitation",
        "income": "Income",
        "hw_all": "Household workload",
    }

    table["Variable"] = table["Variable"].replace(rename_map)

    return table
```

```python
# ##########################################################################
# PART 3: Linear mixed effects models (LMEs)
# ##########################################################################
def fit_lme_models(df):
    dat = df.dropna().copy()

    dat = dat.sort_values(["id", "obs_time"]).reset_index(drop=True)

    formula = (
        "satisfaction ~ obs_time + sex + age_marriage + "
        "cohab + income + hw_all"
    )

    # LME 1: Random intercept only
    lme1 = smf.mixedlm(
        formula=formula,
        data=dat,
        groups=dat["id"],   # random intercept
    ).fit(reml=False)

    # LME 2: Random intercept + random slope for time
    lme2 = smf.mixedlm(
        formula=formula,
        data=dat,
        groups=dat["id"],
        re_formula="~visit",  # add random slope
    ).fit(reml=False)

    return lme1, lme2


def format_lme_fixed(result):
    """Return fixed effects only from an LME model."""
    fe_params = result.fe_params
    fe_se = result.bse_fe
    fe_ci = result.conf_int().loc[fe_params.index]

    df_out = pd.DataFrame({
        "Variable": fe_params.index,
        "Estimate (SE)": [
            f"{est:.3f} ({se:.3f})"
            for est, se in zip(fe_params, fe_se)
        ],
        "95% CI": [
            f"{l:.3f} to {u:.3f}"
            for l, u in fe_ci.values
```

```
        ],
    })
    return df_out.set_index("Variable")


def question3_pretty(df):
    lme1, lme2 = fit_lme_models(df)

    t1 = format_lme_fixed(lme1)
    t2 = format_lme_fixed(lme2)

    t1.columns = ["Estimate (SE)", "95% CI"]
    t2.columns = ["Estimate (SE)", "95% CI"]

    t1.columns = pd.MultiIndex.from_product(
        [["LME 1: Random intercept"], t1.columns]
    )
    t2.columns = pd.MultiIndex.from_product(
        [["LME 2: Random intercept + slope"], t2.columns]
    )

    table = pd.concat([t1, t2], axis=1).reset_index()

    rename_map = {
        "Intercept": "Intercept",
        "sex[T.Male]": "Male (vs Female)",
        "time_index": "Time",
        "obs_time": "Time",
        "age_marriage": "Age at marriage",
        "cohab": "Premarital cohabitation",
        "income": "Income",
        "hw_all": "Household workload",
    }

    table["Variable"] = table["Variable"].replace(rename_map)

    return table


def lme_fit_stats_text(lme1, lme2):
    # lme1, lme2 = fit_lme_models(df)
    return display(Markdown(
    f"""
*Note:* Fixed effects are presented as estimate (SE) with 95% confidence intervals.
*Model fit statistics:*
> LME 1 AIC: {lme1.aic:.2f}, BIC: {lme1.bic:.2f}, Log-likelihood: {lme1.llf:.2f};
```

```
> LME 2 AIC: {lme2.aic:.2f}, BIC: {lme2.bic:.2f}, Log-likelihood: {lme2.llf:.2f}.
"""
))


# ##############################################################################
# PART 4: Model validation
# ##############################################################################
def plot_residuals(model):
    fitted = model.fittedvalues
    residuals = model.resid

    plt.style.use("seaborn-v0_8")
    plt.figure(figsize=(6, 4))
    plt.scatter(fitted, residuals, alpha=0.5)

    plt.axhline(0, linestyle="--")

    plt.xlabel("Fitted values")
    plt.ylabel("Residuals")
    plt.title("Residuals vs Fitted Values")

    plt.tight_layout()


def plot_random_effects(model):
    # Extract random intercepts
    re = pd.DataFrame(model.random_effects).T

    intercepts = re.iloc[:, 0]

    fig = plt.figure(figsize=(6, 4))
    ax = fig.add_subplot(111)

    res = stats.probplot(intercepts, dist="norm", plot=ax)

    # Change colors
    ax.get_lines()[0].set_color("teal")   # data points
    ax.get_lines()[1].set_color("black")  # reference line

    ax.set_title("QQ Plot of Random Intercepts")

    plt.tight_layout()
```